C++ Chapter 6 Arrays and Vectors

Contents

C++ Chapter 6 Arrays and Vectors 1
What is Computer Science? 1
Online Tutorial 2
Tutorial 1: Stack Frames 2
Arrays 5
  Creating Arrays 5
  Accessing Arrays 6
  Arrays and For Loops 6
  For Each Loop 8
Tutorial 2: MoreArrayFun 9
Vectors 12
Tutorial 3: MoreVectorFun 14
Tutorial 4: Vector Average 17
Assignment 1: Array Basics 18
Assignment 2: Vector Basics 18
  Assignment Submission 18

Time required: 90 minutes

What is Computer Science?

Study of data and algorithms

An algorithm is a set of rules or instructions to be followed. A tool for solving problems.

Programming is Problem Solving.

Solve the problem first.

1. Understand the problem
2. Figure out the algorithms and data

3. Write down the algorithms and data in a way we can understand
4. Translate algorithm into computer language

Online Tutorial

Go through the following tutorials before starting the tutorials

- C++ Arrays
- C++ Arrays and Loops
- C++ Omit Array Size
- C++ Get Array Size
- C++ Multidimensional Arrays
- C++ Arrays Video
- C++ Vectors
- Vectors (Tutorial with Try it Live)
- Vectors and Vector Functions (Video)

Tutorial 1: Stack Frames

Stack frames are an area in memory used to store function calls and their local variables. The stack is comprised of several Stack Frames, with each frame representing a function call.

The size of the stack increases in proportion to the number of functions called, and then shrinks upon return of a completed function.

The Stack works on a LIFO (Last In First Out) basis.

Each stack frame contains:

1. The returning line number
2. Any arguments from the called function
3. Storage space for all the function's (automatic) variables
4. Various bookkeeping data

Consider the following program, containing two functions:

```
#include <stdio.h>
// Function prototypes
void firstFunc();
void secondFunc();
int main()
{
    printf("Hello, World! \n");
    printf("Main is the first function in the stack.\n");
    firstFunc();
        printf("The stack will be empty when the program exits.");
    return 0;
}

void firstFunc()
{
    int myInt = 17;
    printf("This is the second function in the stack.\n");
    printf("This function has an int with a value of: %d \n", myInt);
    secondFunc();
    printf("See you later.");
}

void secondFunc()
{
    int yourInt = 42;
    char myChar = 'd';
    printf("This is the third function in the stack.\n");
    printf("This function has an int: %d, and a char: %c \n", yourInt,
myChar);
}
```

You can use Python Tutor to visualize this.

1. Upon program start an initial stack frame is created for main()

2. firstFunc() is called and a new stack frame is created from unused stack memory, containing:

Line to return to = the line after where it was called from in main() = Line 9

Storage space for an int

3. secondFunc() is called and a new stack frame is created from unused stack memory, containing:

Line to return to = the line after where it was called from in firstFunc() = Line 15

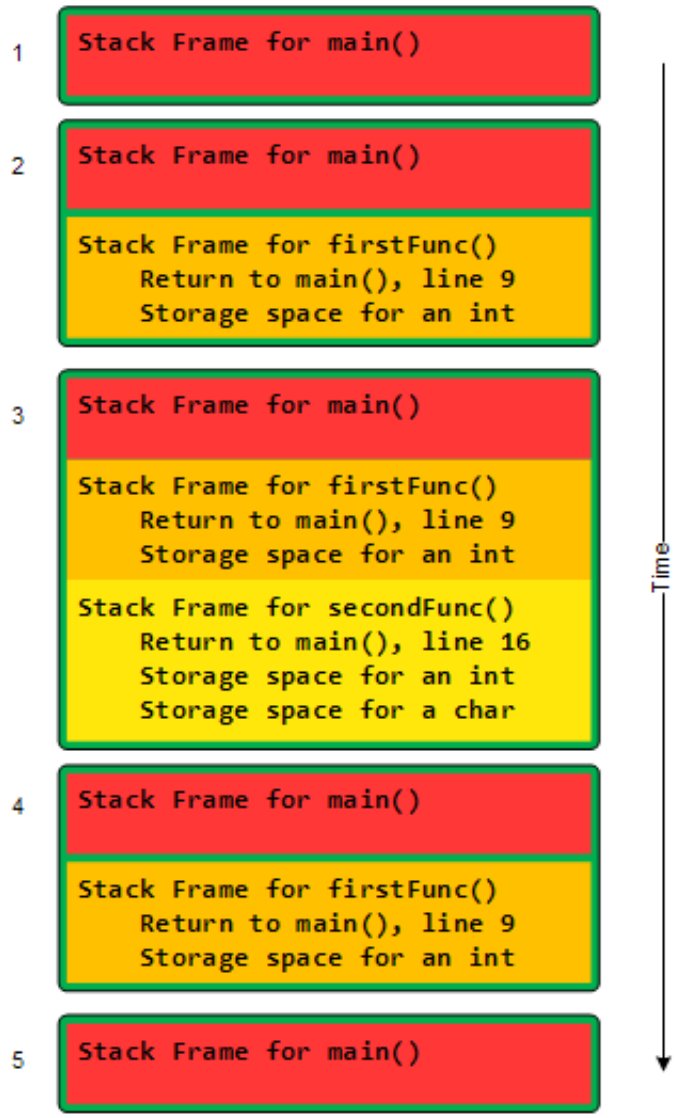
Storage space for an int

Storage space for a char

4. When secondFunc() returns, it's frame is used to determine where to return to (line 15 of firstFunc()), then deallocated and the space returned to the stack

5. When firstFunc() returns, it's frame is used to determine where to return to (line 9 of main()), then deallocated and the space returned to the stack

When main() returns, the program ends.



Example run:

```
Hello, World!
Main is the first function in the stack.
This is the second function in the stack.
This function has an int with a value of: 17
This is the third function in the stack.
This function has an int: 42, and a char: d
See you later.The stack will be empty when the program exits.
```

Please run the above program. Attach a screen shot showing the run.

Arrays

We will learn how to store multiple values of the same type by using a single variable. This language feature will enable you to write programs that manipulate larger amounts of data.

An array is a sequence of values; the values in the array are called elements. You can make an array of ints, doubles, Strings, or any other type, but all the values in an array must have the same type.

Creating Arrays

To create an array, you declare a reference variable with an array type, then create the array itself. Array types look like other C++ types, except they are followed by square brackets ([]).

For example, the following lines declare that **counts** is an "integer array" and **values** is a "double array".

```
int counts[4];
// Constant for size of array
const int SIZE = 5;
values double[SIZE];
```

The number of elements depends on the value of size (at the time the array is created).

You can also declare the variable and create the array with a single line of code:

```
int[] counts = new int[4];
double[] values = new double[size];
```

You can use any integer expression for the size of an array. The value must be nonnegative.

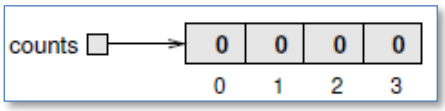
You can initialize an array with a comma-separated sequence of elements enclosed in braces, like this:

```
int[] myArray = {1, 2, 3, 4};
```

This statement creates an array variable, **myArray**, and creates an array with four elements. You don't have to put in the size if you initialize the array when you create it.

Accessing Arrays

When you create an array without assigning values, the elements are initialized to garbage values. The following diagram shows a memory diagram of the counts array.



The arrow indicates that the value of counts is a reference to the array. You should think of the array and the variable that refers to it as two different things. As you'll soon see, we can assign a different variable to refer to the same array, and we can change the value of counts to refer to a different array.

The boldface numbers inside the boxes are the elements of the array. The lighter numbers outside the boxes are the indexes used to identify each location in the array. As with strings, the index of the first element is 0, not 1.

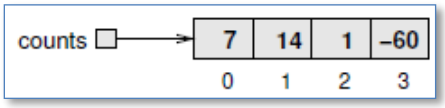
The [] operator selects elements from an array:

```
std::cout << "The first element is " << counts[0] << std::endl;
```

You can use the [] operator anywhere in an expression:

```
counts[0] = 7;
counts[1] = counts[0] * 2;
counts[2]++;
counts[3] -= 60;
```

The following diagram shows the results of these statements.



Arrays and For Loops

For loops are commonly used to iterate through an array one element at a time.

```
/**
 * Name: ArrayForLoop.cpp
 * Written by:
 * Written on:
 * Purpose: Demonstrate array and for loop
 */
#include <iostream>

int main()
{
    const int SIZE = 5;
    // Initialize array
    int counts[] = {1, 2, 3, 4, 5};
    // Iterate through array one element at a time
    for (int i = 0; i < SIZE; i++)
    {
        std::cout << counts[i] << " " << std::endl;
    }
    return 0;
}
```

Example run:

```
1
2
3
4
5
```

The loop variable i is used as the index element to iterate through the array.

For the counts array, the only legal indexes are 0, 1, 2, 3, and 4. If the index is negative or greater than 3, the result is a random integer.

Many computations can be implemented by looping through the elements of an array and performing an operation on each element. Looping through the elements of an array is called traversing or iterating.

```
/**
 * Name: ArraySquared.cpp
 * Written by:
 * Written on:
 * Purpose: Demonstrate array and for loop
 */
#include <iostream>
int main()
{
    const int SIZE = 5;
    // Square each element of an array
    int anArray[] = {1, 2, 3, 4, 5};
    for (int i = 0; i < SIZE; i++)
    {
        // Print original element
        std::cout << anArray[i];
        // Square each element
        anArray[i] = anArray[i] * anArray[i];
        // Print squared element
        std::cout << " " << anArray[i] << std::endl;
    }
    return 0;
}
```

Example run:

```
1 1
2 4
3 9
4 16
5 25
```

This example traverses an array and squares each element. At the end of the loop, the array has the values {1, 4, 9, 16, 25}.

For Each Loop

Traversing arrays is so common, C++ provides an alternative syntax that makes the code more compact. This is like the enhanced for loop in Java.

Consider a for loop that displays the elements of an array on separate lines:

```
const int SIZE = 5;
int values[] = {1, 2, 3, 4, 5};
for (int i = 0; i < SIZE; i++){
    int value = values[i];
    std::cout << value << std::endl;
}
```

We could rewrite the loop like this:

```
int values[] = {1, 2, 3, 4, 5};
// for each value in values
for (int value : values) {
    std::cout << value << std::endl;
}
```

This statement is called a range based for loop, also known as the "for each" loop. You can read the code as, "for each value in values". It's conventional to use plural nouns for array variables and singular nouns for element variables.

Notice how the single line for (int value : values) replaces the first two lines of the standard for loop. It hides the details of iterating each index of the array, and instead, focuses on the values themselves.

Using the enhanced for loop, we can write an accumulator very concisely:

```
int counts[] = new int[100];
for (int score : scores){
    counts[score]++;
}
```

Tutorial 2: MoreArrayFun

Arrays hold multiple values. This example shows a couple of ways to initialize and iterate through an array.

```cpp
/**
 * Filename: MoreArrayFun.cpp
 * Written by:
 * Written on:
 * Demonstration of CPP Arrays
 */

#include <iostream>
#include <string>
#include <array>
int main()
{
   const int ARRAY_SIZE = 10;

   // Creat empty array of Integers
   std::array<int, ARRAY_SIZE> numberArray{};

```

```
18    // Hard code array elements
19    std::string names[4]{"Bob", "Sally", "John", "Ed"};
20
21    std::cout << "Fill array with integers with for loop" << std::endl;
22    // Fill array with for loop
23    for (int i = 0; i < ARRAY_SIZE; i++)
24    {
25       numberArray[i] = i;
26       std::cout << numberArray[i] << " ";
27    }
28
29    // Access individual array elements
30    numberArray[1] = 25;
31    numberArray[3] = numberArray[4] * 2;
32    numberArray[8]++;
33    numberArray[2] += 20;
34
35    std::cout << "\nnumberArray after accesing the array." << std::endl;
36    // Go through array one at a time and print the element
37    // Use the .size() to determine the length of the array
38    for (int i = 0; i < numberArray.size(); i++)
39    {
40       std::cout << numberArray[i] << " ";
41    }
42
43    std::cout << "\nFor each loop with string array" << std::endl;
44    // For Each loop goes through each element in an array
45    for (std::string a : names)
46    {
47       std::cout << a << " ";
48    }
49    return 0;
50 }
```

Example run:

```
Fill array with integers with for loop
0 1 2 3 4 5 6 7 8 9
numberArray after accesing the array.
0 25 22 8 4 5 6 7 9 9
For each loop with string array
Bob Sally John Ed
```

Vectors

In C++, vectors are dynamic arrays that can grow or shrink in size. They are part of the C++ Standard Library and are defined in the `<vector>` header. Here's how you can use vectors in C++:

Include the **<vector>** header at the beginning of your C++ code to use vectors.

```
#include <vector>
```

Declare a vector: You can declare a vector using the following syntax:

```
// Replace 'type' with the type of elements you want to store in the vector,
// Replace 'name' with the desired name of the vector.
std::vector<type> name;
```

For example, to create a vector of integers:

```
// Declares an empty vector of integers
std::vector<int> my_vector;
```

Initialize a vector: You can initialize a vector with elements using various methods, such as:

```
// Initialize with values
std::vector<int> my_vector = {1, 2, 3, 4, 5};

// Initialize with a specific size and default value
// Creates a vector of size 5 with all elements initialized to 0
std::vector<int> my_vector(5, 0);

// Creates a copy of 'another_vector'
std::vector<int> my_vector(another_vector);
```

Accessing elements: You can access elements in a vector using the index (starting from 0) inside square brackets **[]**, just like an array. You can also use the .at() method.

```
// Accesses the first element of the vector Using array notation
int element = my_vector[0];

// Using at() method
std::cout << numbers.at(1) << std::endl;
```

Modifying elements: You can modify elements in a vector by assigning a new value to a specific index.

```
// Modifies the value of the first element to 42
my_vector[0] = 42;
```

Vector methods: Vectors have various built-in methods, such as `size()`, `push_back()`, `pop_back()`, `insert()`, `erase()`, `clear()`, `resize()`, `swap()`, and others. You can refer to the C++ documentation for the complete list of vector methods and their usage.

- **.size() -** Returns the current number of elements in the vector.
- **.capacity() -** Returns the maximum number of elements the vector can hold before resizing.
- **.clear() -** Removes all elements from the vector, leaving it with a size of 0.
- **.pop_back()** - Removes the last element from the vector.
- **.erase(iterator)** - Removes the element at a specified position.
- **.front()** – Returns the first element of the vector.
- **.back()** – Returns the last element of the vector.
- **.insert(new_element, index)** – Inserts a new element at the index.

Here's an example that demonstrates basic usage of vectors in C++:

```
#include <iostream>
#include <vector>

int main() {
    // Declare and initialize a vector
    std::vector<int> my_vector = {1, 2, 3, 4, 5};

    // Access and modify elements
    std::cout << "Element at index 2: " << my_vector[2] << std::endl;
    my_vector[2] = 42;

    // Iterate through the vector with standard for loop
    std::cout << "Vector elements: ";
    for (int i = 0; i < my_vector.size(); ++i) {
        std::cout << my_vector.at(i) << " ";
    }
    std::cout << std::endl;

    // Adds a new element at the end of the vector
    my_vector.push_back(6);
    // Remove the last element of the vector
    my_vector.pop_back();
    // Removes the third element
    my_vector.erase(my_vector.begin() + 2);

    std::cout << "Vector size: " << my_vector.size() << std::endl;
    std::cout << "Vector capacity: " << myVector.capacity() << std::endl;

    return 0;
}
```

Tutorial 3: MoreVectorFun

Vectors are much like arrays. Vectors are mutable, which means that the size of the array can be changed after they are created.

```cpp
/**
 * Filename: MoreVectorFun.cpp
 * Written by:
 * Written on:
 * Demonstration of CPP Vectors
 */

#include <iostream>
#include <string>
#include <vector>

int main()
{
    const int NUM_PEOPLE = 2;
    // Create two parallel vectors
    std::vector<std::string> names;
    std::vector<int> hourlyPay;

    // Variable to store input from user
    std::string tempName;
    int tempHourlyPay;

    // Fill up each vector with information
    for (int i = 0; i < NUM_PEOPLE; i++)
    {
        std::cout << "Please enter a person's name: ";
        // getline gets a string, assigns value to variable
        std::getline(std::cin, tempName);

        std::cout << "Please enter " << tempName << "'s hourly pay: ";
        std::cin >> tempHourlyPay;
        // Consume newline character left over from getting integer
        std::cin.get();

        // Add variable value to the end of the vector
        names.push_back(tempName);
        hourlyPay.push_back(tempHourlyPay);
    }

    // Create separation between input and output
    std::cout << std::endl
              << std::endl;

    // Iterate through vector to display information
    for (int i = 0; i < NUM_PEOPLE; i++)
    {
        std::cout << names[i] << "'s hourly pay is " <<
            hourlyPay[i] << std::endl;
    }

    return 0;
}
```

Example run:

```
Please enter a person's name: Joan
Please enter Joan's hourly pay: 12
Please enter a person's name: Laurie
Please enter Laurie's hourly pay: 12


Joan's hourly pay is 12
Laurie's hourly pay is 12
```

Tutorial 4: Vector Average

```
1 /**
2  * Filename: VectorAverage.cpp
3  * Written by:
4  * Written on:
5  * Average a vector of numbers
6  */
7 #include <iostream>
8 #include <vector>
9
10 int main()
11 {
12     double sum = 0.0;
13     double average = 0.0;
14     const int NUMBER_OF_ENTRIES = 5;
15     std::vector<double> numbers(NUMBER_OF_ENTRIES);
16
17     std::cout << "Please enter " << NUMBER_OF_ENTRIES << " numbers: ";
18
19     // Allow the user to enter in the values
20     for (int i = 0; i < NUMBER_OF_ENTRIES; i++)
21     {
22         // Insert the number into the vector
23         std::cin >> numbers[i];
24         // Keep a running total
25         sum += numbers[i];
26     }
27
28     // Calculate the average
29     average = sum / NUMBER_OF_ENTRIES;
30
31     // Display the results
32     std::cout << "The average of ";
33     for (int i = 0; i < NUMBER_OF_ENTRIES - 1; i++)
34         std::cout << numbers[i] << ", ";
35
36     std::cout << numbers[NUMBER_OF_ENTRIES - 1] << " is "
37               << average << "\n";
38
39     return 0;
40 }
```

Example run:

```
Please enter 5 numbers: 10
25
36
2222
2
The average of 10, 25, 36, 2222, 2 is 459
```

Assignment 1: Array Basics

Write a C++ program that uses an array to store the scores of players in a game. Allow the user to input scores for each player and display the total score at the end.

Assignment 2: Vector Basics

Write a C++ program that demonstrates the basic operations of vectors. Include functions to add elements, remove elements, find the size, and display the elements of a vector.

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.